

ParcanggihIA

Relazione tecnica del progetto di Sistemi Cloud

ParcanggihIA

Stima predittiva della disponibilità di
parcheggio su segmenti stradali reali



Alberto Bocchieri

13 luglio 2026

Indice

1	Introduzione	2
2	Obiettivi e requisiti	2
3	Esperienza utente	2
3.1	Percentuali e colori	4
4	Architettura generale	4
4.1	Perché microservizi	4
5	Servizi applicativi	5
6	Modello dati	5
6.1	Import OSM e override	6
7	Predizione	6
7.1	Uso parsimonioso di TomTom	7
8	Flusso end-to-end	7
9	AI e Text-to-Speech	8
10	Messaging, cache e asincronia	8
11	Modalità di esecuzione	8
11.1	Docker Compose	8
11.2	Kubernetes locale con k3d	9
11.3	AWS reale	9
12	Infrastructure as Code e CI/CD	10
13	Sicurezza e gestione segreti	10
14	Testing e validazione	10
15	Decisioni progettuali	10
15.1	MapLibre invece di Leaflet	10
15.2	Segmenti OSM invece di grandi zone	11
15.3	Fallback simulati	11
15.4	AWS gestito	11
16	Limiti attuali	11
17	Sviluppi futuri	11
18	Conclusione	12

1 Introduzione

ParcheggIA nasce come progetto cloud-native con un obiettivo pratico: aiutare un automobilista a capire dove conviene cercare parcheggio nelle immediate vicinanze, evitando sia una mappa generica sia un semplice elenco statico di aree. La scelta progettuale principale è stata quindi spostare il centro del sistema dal concetto ampio di “zona” al concetto più fine di segmento stradale. Ogni tratto di strada è analizzato come unità indipendente, con un nome via, una geometria, un tipo di sosta e una percentuale stimata di probabilità di trovare posto.

Il risultato è una demo che combina tre aspetti normalmente separati:

- una interfaccia di guida in stile navigatore, costruita con MapLibre GL JS e pensata per essere usata a colpo d’occhio;
- una pipeline dati che parte da OpenStreetMap, integra override locali sulle strisce blu e aggiorna le stime attraverso report, segnali simulati o dati TomTom;
- una architettura a microservizi, eseguibile in locale con Docker Compose, simulabile su Kubernetes locale con k3d e distribuibile su AWS con servizi gestiti.

Il progetto è stato pensato anche per essere dimostrabile senza chiavi esterne. In assenza di TomTom, Nemotron ed ElevenLabs, la piattaforma continua a funzionare con dati OSM reali, predizioni simulate, suggerimenti AI simulati e Text-to-Speech del browser. Le integrazioni reali sono abilitate solo quando vengono fornite le relative API key, senza mai esporle nel frontend.

2 Obiettivi e requisiti

Dal punto di vista funzionale, ParcheggIA deve rispondere a una domanda semplice: “se mi trovo qui, in quale strada vicina ha più senso cercare parcheggio?”. Per rendere credibile la risposta, il sistema non può limitarsi a colorare grandi quartieri. Deve invece lavorare con segmenti piccoli, preferibilmente identificabili con il nome della via, e deve distinguere tra strisce blu, sosta probabilmente libera, sosta limitata o informazioni non note.

Dal punto di vista architetturale, il progetto deve mostrare competenze di Sistemi Cloud. Per questo la stessa applicazione è stata sviluppata in tre modalità operative:

1. **locale**, con Docker Compose, adatta allo sviluppo e alla correzione rapida;
2. **Kubernetes locale**, con k3d, per simulare l’organizzazione cloud senza costi;
3. **AWS reale**, con EKS, RDS, ElastiCache, Amazon MQ, ECR, Terraform, SSM e script di accensione e spegnimento.

Queste tre modalità non sono tre applicazioni diverse. Sono tre modi di avviare lo stesso sistema, con lo stesso frontend, gli stessi microservizi e gli stessi contratti API. Cambiano invece i componenti infrastrutturali sottostanti: in locale PostgreSQL, Redis e RabbitMQ sono container; su AWS diventano servizi gestiti.

3 Esperienza utente

La GUI è stata progettata come interfaccia di guida, non come dashboard amministrativa. Questo ha portato a una serie di scelte concrete: mappa full-screen, pannelli grandi, pochi testi, controlli laterali, dock inferiore, marker leggibili e suggerimenti vocali. L’utente deve poter capire rapidamente se un tratto è favorevole, incerto o critico.



Figura 1: Schermata reale della GUI locale: mappa scura, heatmap continua sui segmenti, marker percentuali, suggerimento AI e controlli stile navigatore.



Figura 2: Dettaglio della visuale di guida: il suggerimento operativo, la posizione utente orientata e i marker di disponibilità restano leggibili senza coprire la mappa.

La mappa è basata su MapLibre GL JS. Questa scelta è stata preferita a Leaflet perché MapLibre lavora nativamente con WebGL, supporta pitch e bearing, gestisce meglio layer vettoriali, heatmap, linee continue e marker sovrapposti. Per una interfaccia di guida, la capacità di inclinare la camera e orientarla nel senso di marcia è fondamentale.

Le informazioni principali mostrate a schermo sono:

- heatmap continua lungo le strade, con colori che indicano la difficoltà stimata;
- marker sui segmenti, con bordo colorato in base al tipo di sosta e percentuale al centro;
- marker dei parcheggi individuati tramite Search API o fallback simulato;
- chip centrale con strada corrente e percentuale;

- suggerimento AI a comparsa dall'alto, letto anche tramite TTS quando disponibile;
- simulazione guida da 500 metri per mostrare il comportamento dinamico senza dover usare un'auto reale.

3.1 Percentuali e colori

La percentuale rappresenta una stima di *parkability*, cioè la probabilità pratica di trovare posto. Non è una misura legale e non sostituisce la segnaletica sul posto. Le soglie usate nella GUI sono volutamente semplici:

Intervallo	Interpretazione	Colore prevalente
< 20%	molto difficile	rosso
20–39%	difficile	rosso/arancio
40–59%	incerto	giallo
60–79%	buono	verde
≥ 80%	favorevole	verde intenso

La heatmap non è generata con punti isolati. Ogni segmento viene campionato lungo la sua geometria e poi visualizzato con un layer morbido e ribbon lineari sovrapposti. Questo evita l'effetto “pallini” e rende la disponibilità visivamente continua lungo la strada.

4 Architettura generale

L'architettura segue un modello a microservizi. Il frontend comunica solo con l'API Gateway; nessuna API key esterna viene mai inviata al browser. I servizi interni si dividono le responsabilità: dati geografici, predizione, sessione utente, ingestion di eventi, amministrazione e AI.

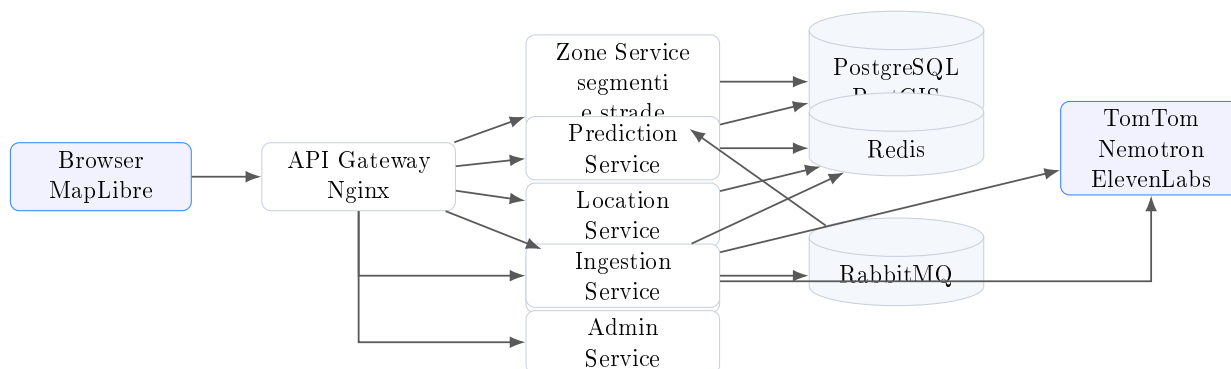


Figura 3: Vista logica dell'architettura a microservizi. Il frontend non parla mai direttamente con provider esterni.

4.1 Perché microservizi

Per una demo piccola si potrebbe implementare tutto in un unico backend. La scelta a microservizi è stata fatta perché il requisito principale non è solo produrre una UI funzionante, ma mostrare una architettura cloud replicabile. Separare i servizi rende espliciti i confini:

- il servizio geografico può evolvere indipendentemente dalla predizione;
- il servizio di ingestion può ricevere dati real-time senza bloccare le API utente;
- il servizio AI può essere spento o sostituito senza compromettere heatmap e navigazione;

- gateway, database, cache e broker hanno ruoli chiari anche quando vengono sostituiti da servizi AWS gestiti.

La controparte di questa scelta è una maggiore complessità operativa. Per contenerla, la repo include script di avvio locale, simulazione Kubernetes, deploy cloud e spegnimento.

5 Servizi applicativi

Servizio	Tecnologia	Responsabilità	Porta locale
frontend	Nginx, HTML, CSS, JS, MapLibre	Interfaccia utente, mappa, heatmap, marker, simulazione guida, TTS browser	8080
api-gateway	Nginx	Reverse proxy unico per API interne e frontend	8000
zone-service	FastAPI, PostGIS	Segmenti OSM, rete stradale, nearest segment, heatmap e override sosta	8001
prediction-service	FastAPI, Redis, PostGIS	Calcolo percentuali, scoring, report, segnali TomTom/simulati	8002
location-service	FastAPI, Redis, RabbitMQ	Sessioni live, posizione corrente, orchestrazione dati vicini	8003
ingestion-service	FastAPI, Redis, RabbitMQ	Ingestion eventi, TomTom parsimonioso, budget e cache	8004
nemotron-service	FastAPI	Suggerimenti AI, fallback simulato, integrazione ElevenLabs TTS	8005
admin-service	FastAPI	Endpoint operativi e diagnostici	8006

6 Modello dati

Il dato centrale è **parking_segments**. Ogni riga rappresenta un piccolo tratto stradale, ricavato da OSM e diviso per incroci o lunghezza massima. A differenza delle vecchie zone demo, questi segmenti sono pensati per sovrapporsi alla rete stradale realmente visibile in MapLibre.

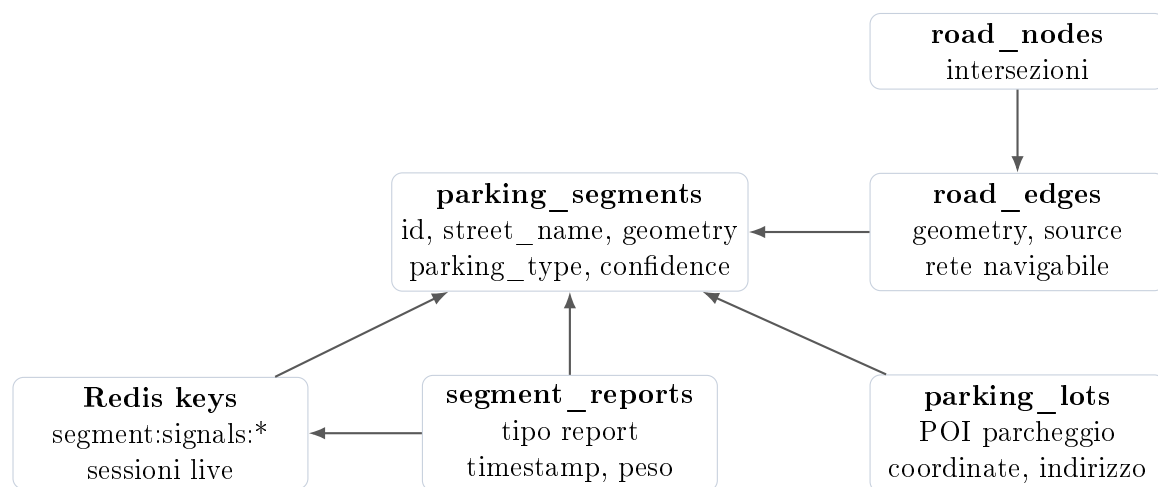
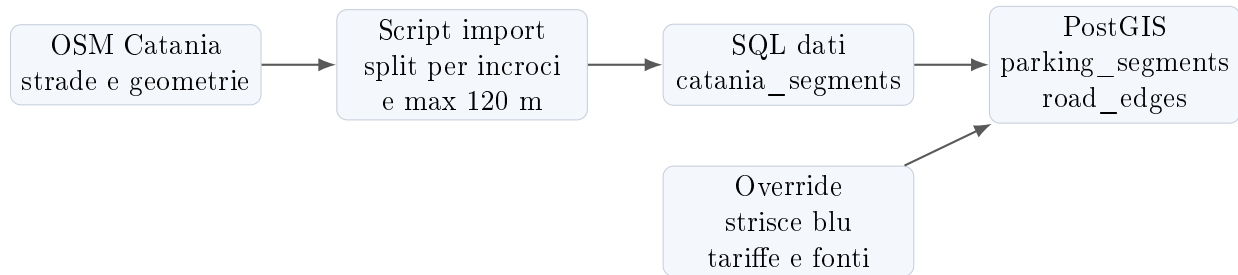


Figura 4: Modello dati semplificato: segmenti per la sosta, rete stradale per la navigazione, Redis per segnali dinamici.

6.1 Import OSM e override

La pipeline offline importa la rete stradale di Catania da OpenStreetMap. I segmenti vengono puliti, spezzati e caricati in PostgreSQL/PostGIS. Successivamente vengono applicati override locali per rappresentare meglio strisce blu, limitazioni e aree note. Questa distinzione è importante: OSM fornisce la geometria stradale, mentre gli override correggono informazioni specifiche sulla sosta che spesso non sono complete nei dati aperti.

Il flusso offline è:



7 Predizione

La predizione non pretende di essere un modello statistico completo. In questa fase è un modello rule-based spiegabile, costruito per essere leggibile e integrabile con fonti real-time. Il punteggio parte da una baseline legata al tipo di sosta e viene corretto da segnali dinamici.

Tipo sosta	Baseline disponibilità	Confidence iniziale
Strisce blu	42%	70%
Probabile libero	48%	52%
Limitato	14%	66%
Ignoto	42%	42%

Le correzioni principali sono:

- report recenti degli utenti, con effetto positivo o negativo sul segmento corrente e sui segmenti vicini;
- segnali TomTom o simulati, salvati su Redis come pressione traffico e pressione evento;
- ora e giorno, perché la disponibilità cambia nelle ore trafficate, di notte e nel weekend;
- distanza dal punto utente, per dare più peso ai segmenti immediatamente vicini.

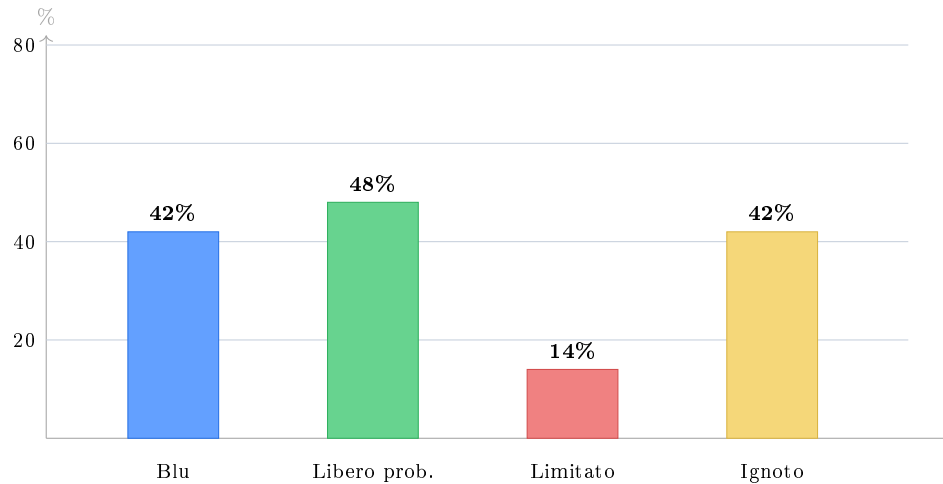


Figura 5: Baseline usate dal modello predittivo prima dei segnali dinamici.

7.1 Uso parsimonioso di TomTom

TomTom viene usato solo backend-side. La GUI chiama endpoint interni; il servizio di ingestion decide se usare cache o fare una chiamata reale. La strategia è volutamente conservativa:

- raggio operativo di circa 500 metri intorno all'utente;
- cella di cache di circa 250 metri;
- TTL minimo di 5 minuti;
- nessuna chiamata a TomTom per singolo segmento;
- budget guard in Redis per evitare consumo eccessivo.

In pratica una nuova zona live può consumare fino a poche chiamate Traffic Flow e una chiamata Traffic Incidents; muoversi dentro la stessa cella non consuma nuove quote. Questo è coerente con l'uso previsto: aggiornare il contesto locale, non interrogare il provider a ogni movimento sulla mappa.

8 Flusso end-to-end

Il seguente esempio mostra il percorso di un dato dalla sua origine fino alla visualizzazione. Si considera il caso in cui l'utente clicchi su una strada o venga spostato dalla simulazione guida.

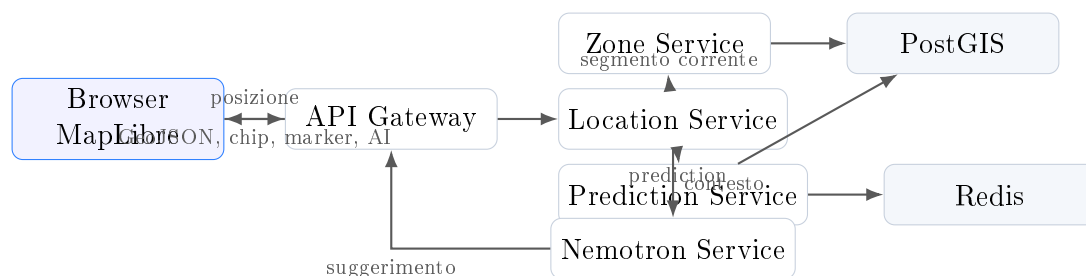


Figura 6: Flusso pratico: dalla posizione utente al rendering della mappa e al suggerimento AI.

Nel dettaglio:

1. il frontend aggiorna la posizione e la invia al gateway;

2. il **location-service** aggiorna la sessione live in Redis;
3. il **zone-service** cerca in PostGIS il segmento più vicino e i segmenti entro il raggio locale;
4. il **prediction-service** calcola la percentuale combinando baseline, report, Redis e segnali dinamici;
5. il frontend aggiorna heatmap, marker e chip centrale;
6. quando serve, il **nemotron-service** genera un consiglio operativo usando il contesto disponibile;
7. il TTS legge il suggerimento, usando ElevenLabs se configurato o il browser come fallback.

9 AI e Text-to-Speech

Il servizio AI è separato dal resto del backend. Questo permette di tenere il sistema funzionante anche senza modello esterno e di non rallentare il calcolo delle predizioni. I suggerimenti non devono essere tecnici: l'utente non deve leggere termini come “stima inferita” o dettagli interni del modello. Il prompt chiede quindi consigli pratici, in italiano, basati solo sui segmenti e parcheggi presenti nel contesto.

Quando sono disponibili le chiavi:

- Nemotron genera il suggerimento testuale;
- ElevenLabs genera una voce più naturale;
- la GUI mostra una card animata che scende dall'alto e si chiude quando il parlato termina.

Senza chiavi, il servizio restituisce una modalità simulata, sufficiente per dimostrare il flusso end-to-end. La scelta è intenzionale: chi clona la repo deve vedere l'esperienza completa anche senza configurare provider esterni.

10 Messaging, cache e asincronia

Redis e RabbitMQ svolgono ruoli diversi. Redis è usato per stato breve, cache, sessioni live, budget e segnali predittivi. RabbitMQ è usato per eventi asincroni, ad esempio quando un report o una ingestion deve influenzare i segmenti senza bloccare la richiesta HTTP principale.

Componente	Uso nel progetto	Alternativa cloud
PostgreSQL/PostGIS	dati geografici persistenti	Amazon RDS PostgreSQL
Redis	cache, sessioni, segnali, budget	Amazon ElastiCache
RabbitMQ	eventi asincroni	Amazon MQ for RabbitMQ
Docker Registry	immagini locali o ECR	Amazon ECR
Kubernetes	orchestrazione container	Amazon EKS

11 Modalità di esecuzione

11.1 Docker Compose

Docker Compose è la modalità di sviluppo più diretta. Un solo comando avvia database, cache, broker, microservizi e frontend:

```
docker compose up -d --build
```

Il primo avvio può essere più lento perché importa i segmenti OSM reali e applica gli override. Una volta completato, l'app è disponibile su <http://localhost:8080>. Questa modalità è la più adatta per valutare la GUI e il comportamento base senza costi cloud.

11.2 Kubernetes locale con k3d

La simulazione cloud locale usa k3d per creare un cluster Kubernetes leggero sul PC. Lo scopo non è migliorare le prestazioni, ma mostrare la stessa forma operativa del cloud: pod, service, config, secret, import dati e port-forward.

```
scripts/cloud_sim_local_up.sh
```

Questa modalità è utile didatticamente perché permette di vedere come l'app viene scomposta in workload Kubernetes senza pagare AWS. La differenza principale è che i servizi gestiti sono simulati da pod locali.

11.3 AWS reale

Il deploy AWS usa Terraform per creare l'infrastruttura e script shell per coordinare build, push immagini, configurazione Kubernetes e smoke test. La scelta dei servizi AWS rispecchia i requisiti del progetto:

- **EKS** per eseguire i microservizi su Kubernetes gestito;
- **RDS PostgreSQL** per persistenza relazionale e PostGIS;
- **ElastiCache Redis** per cache e segnali;
- **Amazon MQ for RabbitMQ** per il broker gestito;
- **ECR** per conservare le immagini Docker;
- **SSM Parameter Store** per parametri e segreti applicativi;
- **Lambda/EventBridge** per programmare lo spegnimento automatico.

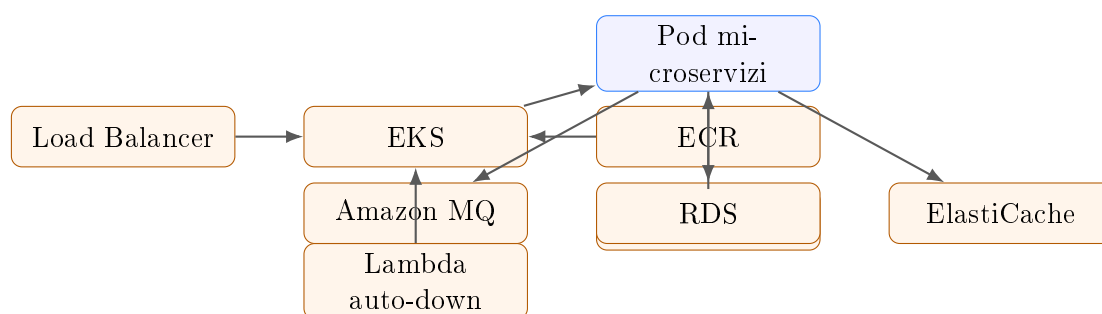


Figura 7: Mappatura cloud reale su AWS. Le risorse costose vengono spente con script e auto-down.

Poiché i servizi AWS hanno costo continuo, la demo cloud è stata pensata per essere accesa solo per periodi brevi. Gli script richiedono conferme esplicite e il comando di spegnimento elimina le risorse create. L'auto-spegnimento programmato a quattro ore riduce il rischio di lasciare l'ambiente acceso.

12 Infrastructure as Code e CI/CD

Terraform descrive l'infrastruttura cloud: VPC, EKS, RDS, Redis, RabbitMQ gestito, ECR, log group, SSM e funzioni di supporto. Questo rende l'ambiente riproducibile su un account AWS diverso, purché l'utente abbia credenziali e quote adeguate. Ansible è presente come parte didattica dell'approccio IaC, ma il flusso principale è Terraform più script shell.

La CI/CD su GitHub Actions verifica il codice e costruisce immagini Docker. Una parte del lavoro è stata rendere le immagini pulite e pubblicabili su ECR, evitando dipendenze da file locali non presenti nel runner. Per il deploy cloud reale, la pipeline può essere estesa con credenziali AWS del repository, ma per sicurezza la demo resta controllabile anche da script locali.

13 Sicurezza e gestione segreti

Un vincolo importante è non esporre chiavi private. La regola adottata è:

- nessuna chiave nel frontend;
- nessuna chiave committata nella repo;
- in locale, chiavi opzionali tramite `.env`;
- in AWS, chiavi opzionali tramite SSM Parameter Store;
- fallback funzionanti quando le chiavi non sono disponibili.

Questo consente a chi clona il progetto di eseguirlo subito, ma permette anche una demo completa con provider reali quando l'ambiente è configurato.

14 Testing e validazione

Il progetto include controlli statici, smoke test e test end-to-end demo. I controlli non sostituiscono test di carico o validazione statistica del modello, ma verificano i contratti fondamentali:

- il frontend contiene le integrazioni attese e non dipende da Leaflet;
- i servizi rispondono agli endpoint principali;
- i segmenti OSM sono importati;
- i contratti API sono coerenti;
- lo smoke test verifica gateway, dati, heatmap e path principali.

Comandi principali:

```
scripts/static_checks.sh
scripts/smoke_test.sh
scripts/e2e_demo_test.sh
```

In cloud, gli script eseguono anche smoke test dopo il deploy, così il link pubblico viene restituito solo quando il frontend e le API risultano raggiungibili.

15 Decisioni progettuali

15.1 MapLibre invece di Leaflet

Leaflet è semplice e maturo, ma è più adatto a mappe 2D tradizionali. ParcheggIA richiede una visuale inclinata, layer lineari continui, marker numerosi e una sensazione di navigatore. MapLibre GL JS è quindi più adatto perché usa WebGL e lavora bene con style vettoriali e trasformazioni camera.

15.2 Segmenti OSM invece di grandi zone

Le zone grandi sono facili da gestire, ma poco utili per un automobilista. Due strade adiacenti possono avere disponibilità e regole di sosta molto diverse. I segmenti OSM permettono consigli più concreti: “prosegui su Via Caronda” è molto più utile di “zona centro”.

15.3 Fallback simulati

Il fallback non è un ripiego marginale, ma una scelta di prodotto e di valutazione. Un progetto didattico deve poter essere avviato da chiunque senza account TomTom, NVIDIA o ElevenLabs. I fallback permettono di verificare architettura, GUI e flussi anche in ambienti isolati.

15.4 AWS gestito

EKS, RDS, ElastiCache e Amazon MQ sono più costosi di un singolo server EC2, ma mostrano meglio il tema cloud-native: separazione dei servizi, responsabilità gestite, scalabilità e sostituibilità dei componenti locali. Per una demo breve, il costo viene mitigato con accensione e spegnimento controllati.

16 Limiti attuali

Il sistema è una demo avanzata, non ancora un prodotto pronto per uso pubblico. I limiti principali sono:

- la predizione è rule-based e non addestrata su storico reale di occupazione;
- TomTom non fornisce disponibilità on-street dei parcheggi con le API attualmente usate;
- il tipo di sosta dipende da OSM e override locali, quindi può essere incompleto;
- la navigazione reale non usa ancora un motore routing completo come OSRM, Valhalla o TomTom Routing;
- la copertura è focalizzata su Catania;
- l'interfaccia è ottimizzata per demo desktop/mobile, non ancora per produzione automotive.

17 Sviluppi futuri

Le evoluzioni più naturali sono:

1. integrare routing reale, con istruzioni e route snapping robusto;
2. raccogliere report utente storici e addestrare un modello predittivo supervisionato;
3. migliorare la qualità degli override sulle strisce blu usando fonti comunali più strutturate;
4. aggiungere osservabilità cloud con dashboard CloudWatch o Prometheus/Grafana;
5. introdurre policy IAM least-privilege per un deploy AWS più vicino alla produzione;
6. separare ambienti dev, demo e prod con Terraform workspace o configurazioni dedicate;
7. aggiungere test visivi automatici su screenshot della GUI.

18 Conclusione

ParcheggIA dimostra come una idea applicativa concreta possa essere trasformata in una architettura cloud-native completa. La parte visibile è una GUI da navigatore con heatmap, marker e suggerimenti vocali; la parte interessante dal punto di vista dei sistemi è il percorso del dato: OSM e override entrano in PostGIS, Redis conserva segnali dinamici e cache, RabbitMQ gestisce eventi asincroni, microservizi specializzati espongono API coerenti e la stessa architettura può essere eseguita in locale, simulata su Kubernetes o portata su AWS.

La scelta di mantenere fallback senza chiavi esterne rende il progetto replicabile. La scelta di supportare provider reali, budget guard e auto-spegnimento rende invece possibile una demo cloud credibile senza perdere controllo su costi e segreti. In questo senso il progetto non è soltanto una applicazione di parcheggio, ma una dimostrazione end-to-end di progettazione, containerizzazione, orchestrazione, integrazione dati, AI applicativa e cloud deployment.